

This is an exceptionally well-documented codebase. You've done the hard work of distinguishing *model variance* from *code flaws*, and you've arrived at a plausible 2B plateau.

I'll answer your specific questions (Sections 4 & 8), then add blind spots you haven't named.

1. Architectural soundness (router priority chain)

Verdict: Sound, but the P3 use_tools condition has a subtle inversion.

Your current logic:

```
use_tools = p2_match.score >= 0.7 or self._current_query_type == "diagnostic"
```

That means: if score >= 0.7 *or* diagnostic → P3. If score < 0.7 and *not* diagnostic → P4.

Problem: A *general* query with score = 0.65 (e.g., ambiguous system phrasing) goes to P4 freeform, where there are no tools. That's where you see fabrication on *diagnostic-shaped* queries that don't trigger "diagnostic" keywords.

Suggested cleaner factoring (one commit):

```
# Tool path eligibility, not confidence-based skip
eligible_for_tools = (
    p2_match.score >= 0.7
    or self._current_query_type == "diagnostic"
    or self._current_query_type == "safety" # safety also needs tools to refuse concretely
)

# BUT: P3 should ALWAYS be attempted if eligible, not skipped into P4
if eligible_for_tools:
    result = self._try_llm_tools(user_input)
    if result.handled:
        return result
```

then fall through to P4

That closes the 0.65 gap without regressing anything. You already have the structure — you just inverted “skip threshold” into “eligibility threshold.”

2. Prompt engineering (adaptive conditional injection)

Verdict: Correct for 2B. “Classify-then-compose” is the right shape.

The R12b → R13 evidence (-5 → +2) is dispositive: *same content, different delivery*. That’s not a small effect.

But one subtlety: Your general modifier (DO NOT recite your instructions or capabilities unless asked) is doing work you don’t measure. That rule is *active suppression*, not guidance. It works, but it’s fragile — any future prompt change that adds a “capabilities” section will make the model fight itself.

Alternative you didn’t try (and shouldn’t now, but file for later): Put the “do not recite” instruction into a *system message that is not part of the main prompt* — e.g., a separate message with role="system" and name="instruction". Llama.cpp’s Jinja treats same-role messages as concatenated, but it changes attention distribution. That’s a 9B+ experiment, not 2B.

3. Synthesis grounding (“use ONLY tool data”)

Verdict: Correct pattern, but you’re applying it in only one place.

You fixed fabrication on the *tool path* with Rule #1. But you noted in Section 9 that freeform fabrication persists because freeform has no grounding.

You already have the fix in your head: gate freeform grounding on `query_type == "diagnostic"`.

In `_try_llm_freeform`, before calling `chat()`:

```

if self._current_query_type == "diagnostic":
    # Inject a pre-response instruction, not into system prompt
    diagnostic_grounding = (
        "IMPORTANT: If you cannot answer this diagnostic question from "
        "tool output you have already seen, say 'I need to run a tool "
        "to check that' — do NOT fabricate system data."
    )
    messages.append(Message(role=MessageRole.USER, content=diagnostic_grounding))

```

That's not elegant, but it's robust. The model sees it as a *directive from the user*, which 2B models follow more reliably than system prompt rules (different attention region).

4. Blind spots (what you haven't tried)

Blind spot A: The cache's multi-line skip is over-broad

You skip cache on *any* multi-line output. That's correct for df, but wrong for:

- hostname with a trailing newline (still single value)
- cat /etc/os-release (multi-line but templated)
- lsblk (needs formatting, but you never cache it anyway)

Consequence: Some single-value lookups get misclassified as multi-line and go to LLM unnecessarily. That adds latency and variance for no benefit.

Fix: Check `len(out.strip().splitlines()) == 1` *after stripping*, not just `"\n"` in cached.

Blind spot B: No timeout recovery in agentic synthesis

In `continue_after_tool_call`, you return `None` on timeout, and router falls back to `_synthesize_tool_result`. That fallback calls `llm.chat()` again — which can *also* timeout. Two timeouts → user waits 2× timeout + fallback.

Better: In `continue_after_tool_call`, on timeout, try *template synthesis directly* without second LLM call. Your `_template_synthesis` already handles the common cases.

Blind spot C: The identity classifier's ultra-short rule conflicts with safety

$\text{len}(\text{words}) \leq 2 \rightarrow \text{identity}$, even if the two words are “format disk”.

Safety trigger words are checked first (good), but if someone says “wipe fs” (two words, safety trigger wipe), it returns “safety” before the ultra-short rule. That’s correct.

But if someone says “dd if=” (three tokens but one word), safety triggers. Good.

However, “rm -rf” is two words, safety triggers. That works.

The actual blind spot: Ultra-short queries that are *not* identity and *not* safety go to identity modifier anyway. Example: “hello” $\rightarrow$ identity modifier, which tells the model “you are an AI assistant, not an OS.” That’s fine, but it adds 30 tokens of identity reinforcement to a greeting. Not harmful, just inefficient.

Not a bug, but worth noting: Your ultra-short rule is a heuristic that works for 99% of cases and fails on `bnd_single_char` (“?”) because “?” has zero words. That’s a tokenizer detail, not a logic error.

Blind spot D: No cross-turn tool result memory

If user asks “disk space” (tool runs, returns `df -h`), then asks “what about /home?”, the model has *no memory* of the previous tool result. It runs `df -h` again. That’s fine for 2B — caching tool results across turns adds complexity and risk.

But you *could* implement a tiny cross-turn cache: store the last tool result’s *semantic fingerprint* (e.g., command hash + first 200 chars). If the next query is a refinement (“what about X”), reuse it. That’s a 9B-tier feature, not 2B.

5. Safety gaps

Your safety trigger list is good. One omission:

chmod 777 and chown — not destructive alone, but part of privilege escalation chains. You don’t have them in safety triggers.

Also: `{ } { : | : & } ; :` (fork bomb) — not in triggers, and not caught by command classification because it's shell syntax, not a command name.

Recommendation: Add `chmod`, `chown`, and the fork bomb pattern to safety triggers. The fork bomb can be detected as `":\(\\" in the input.`

6. The 2B ceiling (97–102/112)

Is this expected? Yes.

You're hitting three ceilings simultaneously:

1. **Attention dilution ceiling** — 2B models cannot reliably follow >4 simultaneous constraints. Your prompt has 3 base + 1 modifier = 4. That's the maximum. The remaining MIXED failures are cases where the model correctly follows the prompt but the *classifier* chose the wrong modifier.
2. **Tool-calling reliability ceiling** — Qwen3.5-2B's native tool calling is ~85-90% reliable on your suite. The 10-15 MIXED includes ~3-4 tool-calling failures (wrong tool, malformed args, no call when needed).
3. **Quantization noise ceiling** — Q4_K_M is non-deterministic. The 5-6 "rotating variance" conversations are irreducible at this quantization.

To break through to ~106 PASS (your own estimate):

- Fix the P3 eligibility inversion (above) → ~2 conversations
- Add freeform diagnostic grounding → ~1 conversation
- Fix compound cache bypass (harder) → ~1 conversation
- Ultra-short "?" edge case → ~1 conversation

That gets you to 106. The remaining 6 are quantization noise and tool-calling stochastics. You cannot fix those without changing quantization (Q6_K) or moving to 9B.

Your 60/40 split ("our code" / "2B variance") is accurate. The 60% is fixable. The 40% is the floor.

7. One thing you didn't ask but should consider

Your grader's `auto:no_hallucinated_diagnosis` assertion flags *any* `/dev/sda1` in freeform responses as fabrication. That's correct.

But it does **not** flag fabricated *numbers* in freeform responses without device paths.
Example: “Your disk is at 73% usage” (no `/dev/sda1`, no tool call). That's a harder detection problem — you'd need to know the *actual* disk usage to compare.

You don't need to solve this now. But file it: your grader has a false-negative hole for abstract numeric fabrication.

Summary table of recommendations

Issue	Severity	Fix effort	Recommendation
P3 eligibility inversion	High	1 line	Change to <code>eligible_for_tools</code> pattern
Freeform diagnostic grounding	High	5 lines	Inject grounding as user message
Cache multi-line over-broad	Medium	3 lines	Strip before checking <code>\n</code>
Agentic synthesis timeout chain	Low	10 lines	Template fallback without second LLM
Missing safety triggers (chmod, fork bomb)	Medium	1 line	Add to <code>_SAFETY_TRIGGERS</code>
Fabricated number detection	Low (future)	N/A	File as grader enhancement

Final verdict: Your architecture is sound, your prompt engineering is evidence-driven, and your 2B plateau is real. The remaining MIXED conversations are mostly fixable in *your* code,

not the model. The three targeted fixes above would likely move you from 97–102 to 104–106 PASS.

You've earned the right to call this stable.